Main Goals

- Improve Data Generation
    - Make data generation more seamless/automated
    - Add weather Tracking
    - Maybe look at a csv file bloat
- Begin Framework For Model's Creation
    - Research how to make the model
    - Begin actually coding model
- As time permits work on SSoT

Data Generation

As it stands, the AI has created a single file (

csv_editing) which takes the csv file given by C.R.I.S and makes a file with negative sampling for the future AI to train off of. The primary issue right now is:

- Weather was randomized as last week was to prove that the general csv file could be made as opposed to proof of its accuracy
- One county has generated around 2.6 millions lines of data

Randomized weather means that the data will be useless for training and is the primary concern for the moment.

The large amount of data is a scalable issue for training. It will make training on history impossible without dedicated expensive equipment and hours that the team doesn't really have. It will also make training in all of Texas (with 1,200 counties or about that many) impossible for us, let alone other states or countries.

Weather

For simplicity the weather has been randomized, but as stated, this makes the data impossible to train with. The same is done for road condition which is a result of weather. The csv_editing needs to have some basis of real historical weather to make training data that can be a reflection of the real world

In the current state of the project (and potentially the final state) weather is assumed to be uniform throughout a county. This is not the most accurate, but for now it will have to be a necessary assumption

Possibility 1
My initial plan was to comb through the initial csv provided from C.R.I.S and find the times in which the weather was not clear. It would then begin creating a new csv file that was arranged as {weatherStart, weatherEnd} and then when creating negative samples, consult this csv file to see if the negative sample fell in these times. If so, then make the weather rainy.

The issue with this idea is the overhead and it makes the road conditions, a direct result of weather, hard to calculate. Additionally, figuring out the exact starting and ending time is hard as the rain would start and end on crash which is not the most realistic and could potentially mess up the AI.

Possibility 2
My second thought was to look at a third party for weather data. This would give some blanket form of information about a county (given the county has all the weather generalized together). It was suggested that some weather station is picked and then data from the station is used. The first recommendation was the NOAA Global Hourly Weather Dataset (https://www.weather.gov/wrh/climate). This seems okay, however it looks like the easiest way to query information on a daily basis is by month. Thus, to create one yearly csv file for a single county, 12 files would have to be made by hand.

This seems to be similar but more actionable:
https://www.ncei.noaa.gov/products/land-based-station/integrated-surface-database. This allows for a csv file to be

downloaded. There is a lot of redundant data and so some exploration of the data will be needed and it will need to be trimmed, but this will work.

Thus the used source will be the second link from NCEA (National Center for Environmental Information). Some city or location will need to be used for the overall county weather. For the sample data I have used DFW airport and the weather station located there. The csv file is organized with these columns:

```
"STATION","DATE","SOURCE","LATITUDE","LONGITUDE","ELEVATION","NAME","REPORT_TYPE","CALL_S
IGN","QUALITY_CONTROL","WND","CIG","VIS","TMP","DEW","SLP","AA1","AA2","AA3","AB1","AD1",
"AE1","AH1","AH2","AH3","AH4","AH5","AH6","AI1","AI2","AI3","AI4","AI5","AI6","AJ1","AK1"
,"AL1","AM1","AN1","AT1","AT2","AT3","AT4","AT5","AU1","AU2","AU3","AU4","AW1","AW2","AW3
","AW4","AW5","AX1","AX2","AX3","ED1","GA1","GA2","GA3","GD1","GD2","GD3","GD4","GE1","GF
1","KA1","KA2","KA3","KA4","KB1","KB2","KB3","KC1","KC2","KD1","KD2","KE1","MA1","MD1","M
G1","MH1","MK1","MV1","MW1","MW2","OC1","OD1","OE1","OE2","OE3","WA1","REM","EQD"
```

This overall is too much data we need: Date (which contains time as well), TMP (Temperature), AA1 (precipitation).

These are structured as the following

- **Date**

<year>-<month>-<day>T<hour>:<Minute>:<Second>

This can be broken up into Date and Hour most likely to make it more readable for humans to validate the data

- **TMP (Temperature)**

+xxxx,x

Where the first number is celsius and the second tells where to round to so the first entry in the data set downloaded:

+0106,1 would be 10.6 degrees celsius. This would be ideal to determine if freezing. I do not know yet if modeling the conditions of the roads is going to be easy or not and so this will allow for checking to see if we need to model ice or snow.

- **AA1 (Precipitation)**

My research has indicated that AA1 is indeed precipitation. Most of the rows inside of the csv file have this row blank, this means there was no rainfall during this time. These entries can likely be removed from the csv file to reduce space taken up by the weather file.

As an example from the csv file:

```
01,0000,9,5
```

Research seems to indicate this is

<Hours reported>, <Precipitation in mm>, <Code>, and <Quality Flag>

The first two are the only ones that matter to us, we want hours reported and precipitation amounts. Any row that does not have this value or a 0 in precipitation amount can be ignored and those that do can be kept. More research about what should be counted as ‘raining’ and how we should model the conditions of the road need to be undergone.

For now, we will model road conditions and and weather based off assumptions of the data

New Functionality in the code

In the backend repo, a new file has been made: csv_weather.py. This is going to be the file that edits weather. The first thing it is set to do is to remove all the columns in the data that are not DATE, AA1, and TMP. From there it is set to remove all the rows that do not contain AA1 information. After that, it is to separate the AA1 information and date information, making it more similar to the training data.

AA1 (Precipitation)

```
01,0000,9,5
```

Date

```
2025-01-01T09:53:00
```

Training data:

```
2025-01-01,1,30,2025,WEDNESDAY,00:00 - 00:59
```

Thus the Date data should be taken and split at the T. From there 2 columns are used: Date and Time where data is the raw 2025-01-01 or whatever equivalent exists. Then time should be converted to hour, taking the hour (09 in this example) and converting it to 09:00-09:59.

The AA1 data should be split at the comma (,) and the first 2 data points preserved while the other 2 data points are discarded. The second data point, precipitation in milliliters, can be put back into AA1 meanwhile, the first attribute which is hours observed can be used to fill in missing hours. After the initial generation, I will do some form of visualization so that I can see if this is a needed addition. If that reads more than one, then duplicate rows are made for each hour so 03,0050,... would become three rows at different consecutive hours all with a recorded 5 mm. Finally we standardize TMP to be a human readable format. Together this create as file that looks like this

```
DATE,TMP,AA1,Time,observation
2025-01-01,6.7,0.0,05:00-05:59,1.0
```

Observation has been for visual elements and it seems there are some data points that do not make sense and look like this

```
2025-01-01,999.9,0.0,05:00-05:59,24.0
```

These are the only ones that seem to observe over multiple hours and given that their recorded temperature is 1000 degrees celsius, they can safely be removed from the training data.

Furthermore, the assumed state of the world can be clear skies and roads. Thus most of the entries that have no precipitation can be removed however, we do have an issue before we can do this which is the conditions of the road.

Conditions of the road

The biggest issue is if ice has managed to clear off of the roads. Even when above freezing, ice stays on the road for some time. Water is also an issue as damp roads exist as well. However, it would be difficult with the data that we have collected to be able to make any kind of accurate predictions about the wet state of the negative samples. Meanwhile, making statements about the road while iced would be easier.

The simplest method at the moment would be to create an ice flag. If the temperature is 0 degrees or less and there is precipitation, make the ice flag set to 1. If the flag is 1 and less than zero, we can assume ice or snow. If above 0 we can assume slush. The flag will continue to be in effect until 5 degrees celsius (about 40 degrees fahrenheit) which is a safe assumption to make.

The logic is, if freezing and raining, all flags from here until the temperature rises above 5 degrees celsius should be flagged as potential ice or winter conditions on the road.

The structure of the file is now

```
DATE,TMP,AA1,Time,ICE_FLAG
```

Error

It was after the meeting on 3/5/2026 that I realized that I had not taken into account three potential types of weather.. I was running off the assumption that all I needed to calculate was clear, rain, and snow. However, I did not take into account Cloudy Skies, Fog, and Hail.

Hail is not something that can be calculated very well from my point of view and it is unlikely that Hail will need to be often reported by the AI, so that can be ignored. This leaves Cloudy and Fog. Both of these might be able to be solved by visibility reporting which is a data column offered by NOAA

Visibility

There is a paper that seems to go over what general levels of visibility are for fog versus cloudy overhead weather. https://www.sciencedirect.com/science/article/pii/S2405844019357652.

To take a direct passage from this paper:
.
1. Dense fog <40 m

2. Thick fog 40 m to 200 m

3. Fog 200 m to 1,000 m

4. Mist/haze 1,000 m to 2,000 m

5. Poor visibility 2,000 m to 4,000 m

6. Moderate visibility 4,000 m to 10,000 m

7. Good visibility 10,000 m to 40,000 m

8. Excellent visibility >40,000 m

This provides a level of detail that is not needed for the project, but it does give us a general level to work with. While there are general statements (mostly from aviation sites) about how cloudy skies can affect visibility, the strict numbers do not seem to exist elsewhere. Thus we will assume

Fog: 1,000 meters or less

Cloudy skies: 10,000 meters or less.

To reflect this data, the NOAA does have a VIS or visibility column with a structure like this:

```
016093,5,N,5
```

The meaning of flags escape my understanding, but the first number is the visibility in terms of meters which allows for easy conversion. The new structure of the WeatherData.csv file is:

```
DATE,VIS,TMP,AA1,Time,ICE_FLAG
```

The next thing to do is to refine the removal of data. Clear weather can still be assumed to be the default, but now we save any rows that have a VIS value less than or equal to 1000.

After making the file, this seems to result in a file with very few (though some do exist) cloudy weather results. The visibility result can be tuned later. For now, this will be acceptable.

From here, we can build a simple csv that details the weather and road conditions

Data to Results

The weather options are:

```
 "1 - CLEAR"
 "2 - CLOUDY"
 "3 - RAIN"
 "4 - SLEET/HAIL"
"5 - SNOW"
"6 - FOG"
```

For these options, the default weather is clear weather. Any hours not referenced can be assumed to be clear. If there is precipitation and the temperature is greater than 0 degrees, then it is rain. If there is precipitation and the temperature is less than 0 degrees, it is snow.

We assume that precipitation overwrites fog and cloudy weather. If there is no precipitation then we use the visibility rules: >= 10000 is cloudy, >= 1000 is fog.

The Road Options are:

```
"1 - DRY"
"2 - WET"
"3 - STANDING WATER"
"5 - SLUSH"
"6 - ICE"
```

The assumed default is dry.

This section is going to suffer from large assumptions. In reality the road conditions are going to linger long after the weather phenomenon is over. However, it would be extremely difficult to track this information, I can assume that most weather inflicted conditions but ice will dissipate quickly.

If there is precipitation and no ICE_FLAG, then we need to decide between wet and standing water. There does not seem to be any formal definition of how much precipitation seems to mean standing water. Additionally, with many roads having draining, standing water is hard to track for any given place. So, we will need to assume that lots of precipitation means standing water. This site (https://www.wawanesa.com/us/blog/hydroplaning-how-to-avoid-it-and-recover-from-it#:~:text=Also%20known%20as%20aquaplaning%2C%20hydroplaning,the%20road%20rapidly%20causes%20slipperiness.) says around .1 inches or 2.5 mm is enough to hydroplane which is probably a safe bet when it comes to standing water.. I am going to round to 3 mm.

Next is slush versus ice. This will be based on temperature. If less than 0 degrees ice. 0-5 (or the cutoff for ICE_FLAG) then it is slush. Not a perfect assumption, but one that will suit the needs of the project.

A file will be created called FinalWeather.csv. WeatherData.csv will provide the raw values and have math calculated on it to produce this file. The FinalWeather .csv file will have a structure like this

```
Date,Hour,Weather,Road
```

Summary

This has taken the weather file which had roughly 8800 lines in the csv down to around 750 rows of usable data, with all the other data being able to be easily assumed or unnecessary

However, the csv_weather.py file works like this:

- **Step 1:** Unnecessary columns are removed from the data. All that is kept is DATE, TMP (Temperature), AA1 (Amount of precipitation), and VIS (Visibility).
- **Step 2**: The DATE and AA1 are separated into DATE (just the day of year), TIME (the hour something happened), and AA1(amount of precipitation). Then AA1's precipitation, the TMP, and VIS are normalized into human readable format
    - AA1 is measured in Millimeters
    - TMP is measured in Celsius
    - VIS is measured in meters
- **Step 3**: ICE_FLAG is created. Ice does not melt automatically or very quickly so the file keeps track of how long ice should roughly remain on the road. ICE_FLAG is set to 0 when temperatures hit 5 degrees or higher.
    - What I assume is debug data is removed here as well, some entries have temps of 999.9 degrees celsius, those are removed as those cannot be real data
- **Step 4:** Any data that does not report precipitation or have an ICE_FLAG will be removed. The exception being if visibility is less than 10000 meters. These are the values that will report some sort of weather condition other than 1 - CLEAR and 1 - DRY which are the default assumed values
- **Step 5**: General calculation is done on the values from WeatherData.csv (my explanation for these values are in the document above) to create FinalWeather which directly states the assumed weather conditions and is ready to be plugged into csv_editing.

Joining Test Data and Weather Data

At present, the csv_editing file randomizes the weather for each of the entries. This was done to simplify the process last week and show that data could be created. Now that we have actual data we can pull from, we need to make sure that csv_editing is using FinalWeather.csv to implement real values onto data.

The final result is a file that has combined all the test data with the 'real data' that we got in terms of weather. As a result, the weather is more realistic.

Looking over pockets of the weather, choosing a single spot to take weather from I still feel is necessary in order to make sure that the process for getting weather doesn't take hours on end (given 254 counties) but this does result in crash data weather being usually different from the negative sample weather. As of yet, I do not know how much this will affect the model's output, but it is a good weakness to track and keep in mind.

Data Size

The biggest issue I am seeing inside of the data right now is the massive ballooning of data size. The initial data that was taken for the crash records of Dallas County had around 50,000 rows of data, which was plenty. However, in the desire to have history for each row, this has ballooned to almost 4 million rows of data. More importantly than the number of rows, although it is a symptom of it, the file for just Dallas county is 454 megabytes or about alf a gigabyte. While Dallas is a large county, there are 254 counties and if all of these files result in similar sizes, we could look to around 127 gigabytes for a year (0.5 * 254). Even a low estimate may be around 100 gigabytes and given the initial goal was years of history (around 5), this could mean half a terabyte of data which is unreasonable to store and would result in unreasonable training size.

Because of this either the scope or the project must change or how data is stored.

Methods

The roads being tracked are at this point simplified to a degree. The average number of crashes is collected from the data and then any roads with less than that are generalized as 'Non Critical Road,' the first option is that a larger number of roads could be lumped into the non-critical roads. This could make the data closer to a fourth of all roads being tracked which would cut the data by a lot.

Another option would be to cut the time slice in half or more with every 2 hours being a sample. This would cut the data size in half immediately. This could not be done alone but could be done in conjunction with something else.

Another way to do this would be to step back and instead of tracking roads at all, we could just generalize based on the counties/cities. This would need to result in a large time slice or even just x negative sample an hour. This would dramatically cut down on the amount of data needed.

One method that was mentioned to me was creating x negative samples per positive sample. So, for every crash, generate 5 non-crashes. This would work well to cut down data and would result in somewhere around 250,000 thousand rows which would be well under a gigabyte. This is the best version of this I have seen so far, but I do worry that this will not accurately depict the ratio of crashes to non crashes. It would give the impression that for every crash there are at least 5 non-crashes. This isn't wrong but I do worry this removes the ability for the AI to understand some roads are more dangerous than others

I have thought up a hybrid method from the above example and time slices which will be detailed below.

<u>There will be a team meeting on 3/5/2026 where I will present my idea and see what is changing if anything</u>

Hybrid Idea

At present the largest issue with the time sampling is that it expands very quickly. With 8,760 samples per road, this means that roads with very little action still take up a large amount of space. In essence roads with 12 yearly crashes have the same amount of crashes as roads with almost 4,000 crashes (per the Dallas County 2025 crash report). My hybrid idea is to create x negative samples per positive sample with a maximum and a minimum. For example, 5 negative samples per positive sample, 50 samples minimum, 8,800 maximum. This would mean that the smallest roads would only have 50 samples total while the largest roads could hit the maximum. Given the rate of fall off in terms of roads with crashes this would likely save a lot of data. The only issue to the plan is 'Non Critical Roads' but they could just have negative samples completely mixed in with the positive samples.

I imagine that this would reduce the amount of less than that of the static, create 5 negative samples per positive sample, but it would reduce space and be scalable to other counties. It would also be able to indicate which roads generally have more crashes by implementing this maximum.

The weakness of this idea is that the numbers used would need to be tailored to a state which is difficult. For example, while Texas is large a state like say Idaho (random example) may never get near the maximum mark and so a majority of their roads would hit the minimum threshold. It also means that while the larger roads near the maximum would be covered well, the roads between the minimum and the maximum would have their respective weights of crashes screwed up.

I will present this during the meeting on 3/5/2026 and see what the team has to say about it and we will move on from there. It is unlikely that I will begin making a model this week, but I will do my best to get the data done and in a scope that is usable with weather. This will put me behind and I may need someone for next week to get the phase 2 goal done.

3/5/2026

The meeting happened and it was mostly just a check in before the meeting to discuss questions. The issue that I had about the general size of the files and the scalability seems to have been brushed aside. The general consensus seems to be that so long as Dallas County works for now, the data can be modified later.

Thus, for now reducing the amount of data generated will not be in my main view.

However, one thing that did come up was that the csv for editing data was not very usable and made little sense to those who had no experience. I need to make sure that it is fully automated, the csv_editing and csv_overview work together as there are functions from editing that are hard coded from inputs from overview. Additionally, my documentation should be better as this should be able to be queried by anyone at any time so that data can be accessed by anyone on the team.

Single Source of Truth

My intent until the meeting is to work on the single source of truth to the best of my ability as it is not yet done

Overview

From the main goals, my only real improvement on the week was to improve data generation. The weather now works but that tools many hours more than I thought it would. I made some progress on making the process more streamlined, but making the values non-hard coded in has yet to be done. Fortunately, barring any need to completely redesign the the data generation, next week should have the data generation be a small part of the project

The AI model was not able to be researched. The SSoT was not complete and that seemed to be a bigger issue than beginning the research.

Next Week Tasks

My intent for next week will be to work on the data, ensuring no hard coded values (unless the file used is generated by the data creator). Furthermore, documentation will improve for the entirety of the file.

The AI model and using PyTorch needs to be researched. While I understand what it is, I do not yet know how to make a model or what goes into it.

These are both large tasks at present and if new tasks appear during the meeting at 3/6/2026, then I will reference them in my week 4 report.

Overall, this week took longer than I thought I am behind on what my tasks are, but I think my group will help me in compensation for that in the coming week.